History of C++

1962 - Simula I developed.

↓

1967 Simula 67 was the first - object

↓

1967 Simula + 1970 C

↓ 1979 Stroustrup.

C with Classes.

→ 1983 — C++

→ 1985 — standard ANSI

C++ → 1989 Compiled
→ C Compatible
→ High Performance
→ OPS.

terminology

object, member function, Data members, Class, Attributes

Basic of C++

→ Compilation
→ The Preprocessor.
→ declaration & definition
→ main function.

Memory Layout

1. Text → Code
2. Static → Variable
3. Heap → dynamic
4. Stack → function & Subroutine

The preprocessor →

→ runs before the Compilation
→ modify the source code
→ Processes preprocessor instruction
→ strips out Comments

→ # include
  ↳ insert a file
    - #include <iostream>
    - #include "abc.txt"

→ # define
  ↳ define a macro

→ #ifdef, #ifndef, #endif
  ↳ check to see if a macro has been defined

```
# include <iostream>
Using namespace std;
```

Example =>

(1)

```
#define versionnumber "1.0Beta"
int main()
{
cout << "running version" << versionnumber << endl;
return 0;
}
```

(2)

```
#define PI 3.14
int main()
{
double radius = 4.0;
double area = PI * radius * radius;
cout << Area of a Circle" << area;
}
```

(3)

```
#include <iostream>
using namespace std;
#define PRINT_RECTANGLE (width, height)
for (int i=0; i<height; i++)
{
for (int j=0; j<width; j++)
{
cout << "*";
}
cout << endl;
```

Declaration & Definition =>

- introduce a name or something.
- tells what kind of thing it is
- gives programmer enough information to use.

Definition

- introduce or repeate a name for something
- tell what kind of things,
- tell what value it has/or how it works
- give compiler enough information to generate this and assign it an address.

main() function →

- start of every programme in C++
- return statement return back value of its type.

<< - insertion operator

>> - Extraction operator.

Token, keywords & Identifiers

- a smallest individual unit.
  - it is keyword identifiers of, constant, etc.

keywords
- implements specific C++ language tasks.

Identifier:- refers the name of variable, function, array.

Data types in C++

- User defined
  - class
  - structure
  - union
  - Enum
- Built in
  - char - 1B
  - int - 4B
  - unsigned int - 4
  - short int - 2
  - long int - 8
  - float - 4
  - double - 8
  - Bool - 1
- Derived
  - Array
  - String
  - Pointer

Structure → Heterogeneous

↳ Collection of variable of different types under a single name for better visualization.

①

```
Struct Person
{
    Char name[50];
    int age;
    float Salary;
};

Person bill;
bill.age = 50;
```

②

```
int main{
    Person P1;
Cout << "Enter hum name" << endl;
Cin >> P1.name;
cout << "Enter age" << endl;
```

```
Cout << "Enter Salary" << endl;
Cin >> P1.Salary;
Cout << "Display information" << endl;
Cout << "Name:" << P1.name << endl;
Cout << Age" << P1Age << endl;
Cout << "Salary" << P1.Salary;
}
```

Union ⇒

↳ that uses the same memory as other object from the list of object.

```
Union Emp
{
    int num;
    double sal;
};
```

Example:

```
Union test
{
  int x;
 Char arr[8];
  int y;
};

 int main()
{
  Cout << Sizeof(test) << endl;
      return 0;
}
```

Enum:: Enum is also known as Enumeration is a user defined data type that consist of integral constants.

→ it provide a way to define and group integral constants.

→ enum key words is used.

→ enum are generally used when we expect the variable to select one value from possible set of values.

Ex.

①
```
#include <iostream>
using namespace std;
enum year { Jan=0, Feb, Mar, APr, May, Jun, Jul, Aug
    Sep, Oct, Nov, Dec };
int main ( )
{
   int i;
for (i= Jan;  i<=Dec ; i++)
    {
        Cout << i << " " <<;
    return 0;
}
```

O/P → 0 1 2 3 4 5 6 7 8 9 10 11 12.

②
```
enum Colors { red=5, black};
enum Suit { heart, diamond=8, Spade=3, Club};
 int main ( )
{
 Cout << red << black;
 Cout << heart << diamond << Spade << Club;
 return 0;
},
```

O/P → 5, 6

0, 8, 3, 4

3.

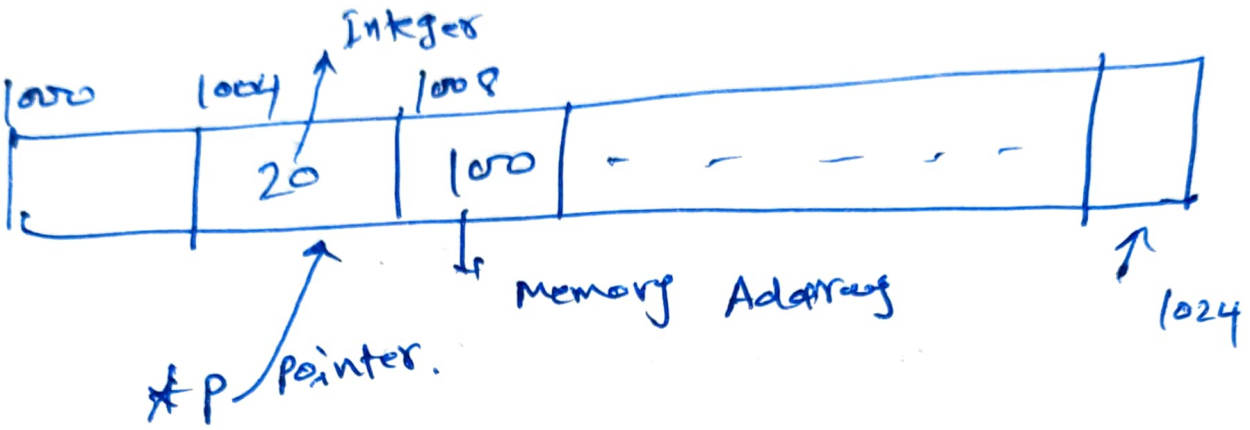
```
#include <iostream>
using namespace std;
enum cat { temp=7, tempp, temppp };
int main()
{
    float div = 18;
    div/= temppp;
    cout << div << endl;
    return 0;
}
```

O/P → 2.

Pointers ⇒

A Pointer is a Variable used to Store the Address of a memory cell. We can use the Pointer to refrence this memory cell.

data type *Var-name ;



int *P ;

*p = 20 ;

Operation on Pointers ⇒

- Define a Pointer,
- Assign the address of a Variable to pointer,
- the Value of the address available in the Pointer Variable and this is done by using the unary operator * that return the Value of the Variable located at the Address Specified by the Operand.

int 21; → Variable declaration

int *P; → Pointer Declaration

ip= &21; → Address Assignment;

*ip = 10; → Value assignment to 21.

Cout << 21 → Print 21 Value

Cout << &P → Print the address of 21

Cout << *P → Print the 21 Value.

Pointer to Pointer ⇒ A Pointer who having the address of another Pointer.

```
int a;
int *P;
int **q;
```

P = &a;

q = &P;

Cout << a ; → Print a Value.

Cout << *P ; → Print a Value.

Cout << **q → Print a Value.

Example:

```
#include <iostream>
using namespace std;
int main()
{
```

① `int a=100, b=88, c=8;`

`int *p1=&a, *p2, *p3=&c;`

`p2 = &b;`

`p2 = p1;`

`b = *p3;`

`*p2 = *p3`

```
cout << a << b << c;
} return 0;
```

O/P → 8, 8, 8..

② `int a=120, b=80, c=40;`

`int *p1=&a, *p2=&b, *p3=&c;`

`*p1 = *p2 + *p3;`

`p2 = p1;`

`*p2 = *p3 + 10;`

`p3 = &b;`

`*p1 = *p2 + *p3;`

`b = *p1 - *p3;`

`*p3 = *p1 - b;`

```
cout << a << b << c << endl;
```

o/p → 130 80 40.

(3)

```
int a=50, b=30, c=20;
int *p1=&a, *p2=&b, *p3=&c;
*p1 = *p2+*p3;
p3 = p1;
*p2 = *p3-10;
*p1 = *p2+*p3;
p2 = p1;
*p2 = *p1-a;
Cout << a << b << c << endl;
```

NULL Pointer: A Pointer that is assigned NULL value called a NULL pointer.

```
int main()
{
    int *ptr = NULL;
    Cout << "the value of ptr is" << ptr;
    return 0;
}
```

Pointer & Array =>

```
int main( )
{
  int var[3] = {10, 20, 30};
int *ptr ;

 ptr = var;
for (int i=0 ; i<3 ; i++)
{
  Cout << *ptr << endl;
}   ptr ++;

return 0;
}
```

Array of Pointers: =>

```
int *ptr[MAX];
```

→ Pointer as an array of MAX Integers Pointer

→ Each element now holds a pointer to an integer value.

Example:-

```
int main( )
{
  int var[MAX] = {10, 100, 200};
```

```
    int *ptr[MAX];
    for (int i=0; i<MAX; i++)
    {
        ptr[i]= &var[i]
    }
    for (int i=0 ; i<MAX', i++)
    {
        Cout <<"Value of var["<<i<<"]=";
        Cout << *ptr[i]<<endl;
    }
        return 0;
}
```

```
int *ptr[MAX];
for (int i=0; i<MAX; i++)
{
    ptr[i] = &var[i]
}
for (int i=0; i<MAX; i++)
{
  Cout << "Value of var[" << i << "] = ";
  Cout << *ptr[i] << endl;
}
  return 0;
}
```

Pointer to Pointer Example:

```
int a=10, b=20, c=30;
int *p1= &a;
int *p2= &b;
int **pp1 = &p1;
int **pp2 = &p2;
**pp1 = **pp2 + c;
   p2 = p1;
   pp2 = &p1;
```

```
**pp2 = a+b;
p1 = &c;
**pp1 = *p2+5;
cout << a << b << c;
```

O/P → 75 20 30

(2) int a=10; b=20, c=30;

```
int *p1=&a; int *p2=&b; int*p3=&c;
int **pp1=&p1;
int **pp2=&p2;
int **pp3=&p3;

**pp1 = **pp2 + **pp3;
*pp2 = *p1 - *p3;
pp3 = pp1;
**pp3 = *pp1+*p2;
p2 = p1;
*pp2 = *p3 + *p1;
pp1 = &p3;
*pp1 = *p2-*p1;
pp1 = &b; ——→ Error.
**pp1 = *p1 + *p2;
cout << a<<b<<c;
```

→ 50, 40, 40

Array & pointer →

(1)

```
#include <iostream>
using namespace std;
int main()
{
  const int i = 20;
  const int * const ptr = &i;
  (*ptr)++;
  int J = 15;
  ptr = &J;  → Compile time Error.
  cout << i;
  return 0;
}
```

(2)

```
int main()
{
  int num[5];
  int *p;
  p = num;
  *p = 10;
  p++;
  *p = 20;
  p = &num[2]
```

```
*P = 30;
P = num+3;
*P = 40;
P = num;
*(P+4) = 50;
for (int i=0; i<5; i++)
{
  cout << num[i] << endl;
}
```

(3)

```
#include <iostream>
using namespace std;
int main()
{
  int arr[] = {4, 5, 6, 7};
  int *P = (arr+1);
  cout << *arr+10;
  return 0;
}
```

Return (4)

```
#include <iostream>
using namespace std;
int main()
```

(8)

```
{
    int arr[] = {5, 10, 15, 20, 25, 30, 35};
int *p = arr+1;
int *q = arr+5;
int x = *p++ + *--q;
int y = *(p+2) - *(q-1);
p = p+1;
q = q-2;
int z = *p + *(q+1);
cout << x << y << z;
cout << *(arr + (*p/5)) + *(q-1);
return 0;
}
```

Reference variable ⇒

① Example.

```
int main()
{
    int x = 10;
    int &ref = x;
    ref = 20;
```

```
Cout << "a = " << a << endl;

a = 30;

Cout << "ref value" << ref;

return 0;
}
```

Output

a = 20
ref = 30

Example 2 :-

```
#include <iostream>
Using namespace Std;
int main()
{
    int a = 32 , *Ptr = &a;
    Char Ch = 'A';
    Char &Cho = Ch;

    Cho += a;
    *Ptr += Ch;

Cout << a << Ch << endl;
    return 0;
```

O/P → 129, 9.

Refrence & Pointer Variable;

① String hello = "Hello";

↑ String is Class

hello is object and it occupy some computer memory.

②

String * hello_ptr = &hello

↑ Now it is a String pointer which hold the address of a String Variable and occupied 4 byte in memory

③ String &hello_ref = hello;

↑ this is a refrence to a String. i.e. hello_ref Variable is assigned the memory address of object hello automatically.

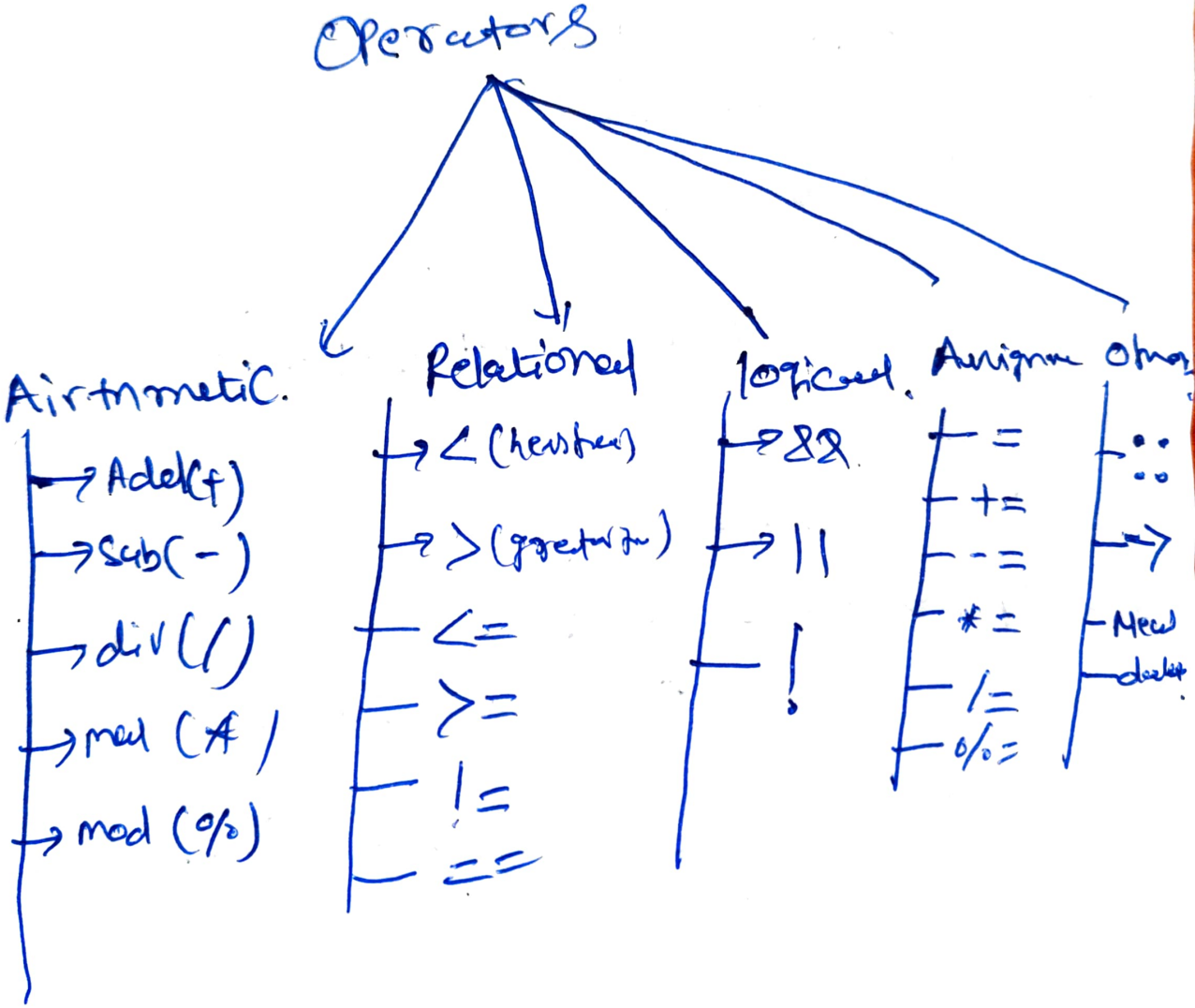
⇒ Refrence and Pointers Variable Stored the memory location of other variable.

⇒ Pointers in C however refrence added in C++.

⇒ Reference Variable do not occupied the memory.

Operators in C++.

- input Operator Cin.
- Output Operator Cout.



Control structure

- Loop
  - do while
  - while
  - for
- Switch
- if else.
- Continue
- Break.

Abstract data type

- It is a class for object whose behavior is defined by a set of value and a set of operations.
- the definition of ADT only mentions what operation are to be performed but not how these operation will be implement.
- it is called abstract because it gives an implementation- independent view.
- the process of providing only the essential and hiding the detail is known as Abstration.

~~Pre and Post Increment Operator~~

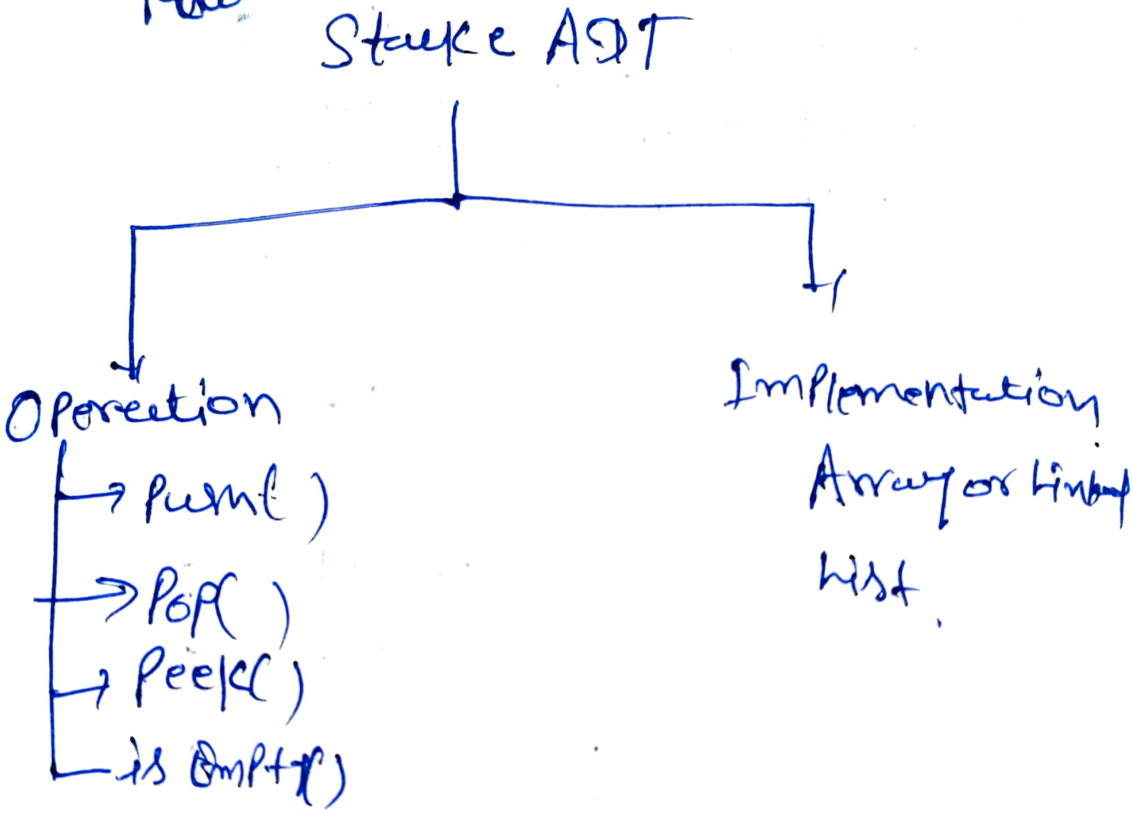
Example: (A) Stack, Queue, List, Set

(B) map/dictionary, Priority Queue.

ADT – What the data structures does.

Data Structure: – How it is implemented.

Now



* the stack does not know whether it is implement with array or linked list.

Dynamic memory management ⇒

→ the memory Allocated at run time according to the program requirment.

→ New → is used to allocate the memory.

↳
```
int *p;
p = new int;
*p = 25;
Cout << p
```

→ New with initialization.

↳
```
int *p = new int (25);
Cout << *p;
```

→ Delete Operator to deallocated the dynamic memory.

```
delete p;
```

Dynamic Array.

```
int n;
Cout << "enter size n" << endl;
Cin >> n;
int *arr = new int[n];
for (int i=0; i<n; i++)
{
    cin >> arr[i];
}
delete[] arr;
```

Example →

①
```
Class Line
{
  Private:
    double length;
  Public
    Void setlength(double l);
    double getlength(void)
};

Void Line::setlength(double l)
{
  length = l;
}

double Line::getlength()
{
  return length;
}

int main()
{
  Line line;
  line.length = 20;
  Cout << line.getlength << endl;
  line..Setlength
  Cout << line.getlength << endl;
  return 0;
}
```

O/P → Compile time error.

Example. (1) Write a program to calculate simple interest
Hide the data elements of the class using private keyword

```
#include<iostream>
using namespace std;
Class interest
{
  private:
float P_amount;
float rate, Period, interest, float t_amount;
Public:
void in( )
{
  Cout << Principal amount; rate, and year"<<endl;
  Cin>> P_amount >> rate >> Year;
  interest = P_amount * Period * rate/100;
  t_amount = P_amount + interest;
}
Void_ Show ( )
{
Cout << P_amount<<total amount << endl.
}
```

```
int main( )
{
    interest r;
    r.in( );
    r.show( )
    return 0;
}
```

(7) An electricity board charges the following rates to domestic users to discourage large consumption of energy:

For the first 100 units: 7/- Per unit.

Beyond 100 units: 7.5 Per unit.

if the total cost of the electricity consumption is more than the 250.00 then an additional surchage of 15% is added to the diffrence else 10% is added to the diffrence. define a class Electricity in which all the data members required Private member and Create the member function Count_bill( ) as a Public for the Computation of the total bill of Electricity Consumption

if the total consumption of the electrity iby the customrare is 253.

Array with in the class:-

```
Class array
{
   int regno[10];
};
```

Array of object:

```
Class Array a[10];
```

Example:

```
Class Details
{
  Private:
  Int Sal;
  int id;
Public:
Void Getname()
{
Cout << "Enter Salary" << endl;
```

Passing & Returning Object.

```
Class demo
{
    int num;
Public:
    Void input (int a)
    {
      num = a;
    }
  Void  Copy (demo);
  Void  Show()
{
   Cout << num << endl;
}
  Void demo :: Copy (demo1)
  {
     num = d. num ;
  }
Void main ( )
{
  demo  d1, d2;
  d1. Input(20);
```

```
demo Copy(Void)
{
  demo temp;
  temp.num = num;
 return temp;
}
```

```
    d2 . Copy(d1);
    d1. show( );
    d2 . show( );
}
```

```
d2 = d1. copy ( );
```

Methods

- Similar to function
- Define inside the Class.
- Can be Private, Public, Protected.
- Can access by only object.
- Operate on the data within the Class.
- Each object has its own stack.

Static member: are those that persist even after control return from the functions.

Static member function

- Placing the static keyword.
- must define inside the Class and Can not separate declaration and definition.
- one for a Class and Can access by object.
- It Can also be called by object.

Static Data member

- only one copy of the static data members in the Class.
- All object share the same copy
- Always initialize with zero.

Static member function

```
Class demo
{
 Public:
 StaticVoid Show()
  {
   Cout << " demo of static "<<endl;
  }
};
 Void main()
{
  demo :: Show();
}
```

~~demo d1~~

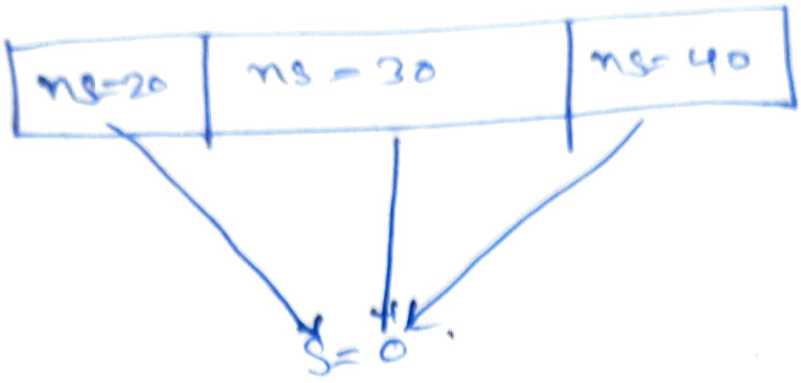
```
demo d1;
d1.Show();
```

↳ Also possible

Static member function

```
Class demo
{
   Static int s;
   int ns;
}
```

demo d1, d2, d3;



Example: (1)

```
Class number
{
   Static int C;
    int K;
Public:
   Void Zero()
   {
       K=0;
   }
   Void  Count()
   {
      ++C;
     ++K;
Cout << C << endl;
Cout << K << endl;
```